

AGEX Canonical Resource & Schema Specification

S-03. AGEX 표준 Resource 및 Schema 통합 명세

문서 등급: Core Implementation Specification

상위 문서: AGEX Master Specification

적용 대상: Core / Runtime / Agent / Workflow / Knowledge / Memory / Plugin / Model / API / SDK / DB / Event / Console

상태: 공식 기준안

목적: 실제 코드·API·DB·SDK 생성 기준 확정

1. 문서 목적

본 문서는 AGEX에서 사용되는 핵심 Resource의 공통 구조와 Domain별 Schema를 하나의 Canonical Contract로 정의한다.

본 문서의 가장 중요한 목적은 다음과 같다.

사람이 읽는 30종 문서의 개념을 실제 코드가 사용할 수 있는 Resource Model과 Machine-readable Schema로 변환한다.

앞으로 다음 구현물은 본 문서를 기준으로 생성되어야 한다.

- Type Definitions
 - JSON Schema
 - API Request / Response
 - SDK Model
 - Event Payload
 - Database Logical Model
 - Validation Rule
 - Console Form
 - Contract Test
-

2. Source of Truth 관계

AGEX의 Schema Source of Truth는 다음 순서를 따른다.

MASTER.md

↓

Canonical Resource & Schema Specification

↓

Machine-readable Schema

↓

API Contract

↓

Generated SDK Types

↓

Implementation

따라서 실제 코드가 본 명세와 다를 경우 코드가 아니라 명세 또는 Schema를 먼저 수정해야 한다.

3. Canonical Schema Repository

권장 구조:

/specs/schemas/

|

├─ common/

| └─ resource-metadata.schema.json

| └─ resource-reference.schema.json

| └─ pagination.schema.json

| └─ error.schema.json

| └─ event-envelope.schema.json

- |
- |— tenant/
- |— identity/
- |— runtime/
- |— agent/
- |— workflow/
- |— knowledge/
- |— memory/
- |— plugin/
- |— model/
- |— governance/
- |— audit/
- |— usage/

4. Schema URI 규칙

각 Canonical Schema는 고유 URI를 가진다.

예:

agex://schema/common/resource-metadata/v1

agex://schema/agent/agent-resource/v1

agex://schema/runtime/execution/v1

Schema Version은 Resource Version과 별개다.

5. Schema Version

Schema Version은 Contract Version을 의미한다.

예:

Agent Schema v1

Agent Resource Version 17

두 개념은 다르다.

6. Resource의 최상위 구조

AGEX Resource의 기본 형태를 다음으로 정의한다.

```
{  
  "api_version": "agex/v1",  
  "kind": "Agent",  
  "metadata": {},  
  "specification": {},  
  "status": {}  
}
```

7. api_version

Resource Contract Version.

예:

agex/v1

이 값은 REST API Version과 반드시 동일할 필요는 없다.

8. kind

Resource Type을 나타낸다.

예:

- Tenant
- Agent

- Workflow
- Execution
- KnowledgeCollection
- Memory
- Plugin
- Model
- Policy

Case Convention은 PascalCase를 사용한다.

9. Resource Metadata

공통 Metadata는 다음을 기준으로 한다.

필드	타입	필수	설명
id	string	Y	불변 Resource ID
name	string		조건 Human-readable name
scope_type	enum	Y	PLATFORM / TENANT
tenant_id	string/null		조건 TENANT Scope일 때 필수
version	integer/null		조건 Published Resource Version
revision	integer	Y	Draft Revision
lifecycle_state	string	Y	Resource Lifecycle
owner	ResourceReference		조건 Production Resource 권장/필수
labels	map<string,string>	N	검색/분류
annotations	map<string,string>	N	비실행 Metadata
created_at	timestamp	Y	UTC

필드	타입	필수	설명
updated_at	timestamp	Y	UTC
created_by	PrincipalReference	Y	생성 Actor
updated_by	PrincipalReference	Y	마지막 변경 Actor

10. Resource ID

ID는 불변이다.

형식:

{prefix}_{opaque-id}

권장 Prefix:

Resource	Prefix
Tenant	ten
User	usr
Membership	mbr
Agent	agt
Workflow	wfl
Execution	exe
Task	tsk
Knowledge Collection	knc
Knowledge Source	kns
Memory	mem
Plugin	plg
Plugin Installation	pli

Resource	Prefix
Model	mdl
Provider	prv
Policy	pol
Approval	apr
Event	evt
Artifact	art
Operation	op
Audit Event	aud

Prefix는 Debugging 편의를 위한 것이며 Security 기준으로 사용하지 않는다.

11. Scope Type

공식 Enum:

PLATFORM

TENANT

PLATFORM

플랫폼 전체 Scope.

TENANT

특정 Tenant Ownership.

TENANT이면 tenant_id가 필수다.

12. Resource Reference

공통 구조:

{

```
"type": "Agent",  
"id": "agt_123",  
"version": 4  
}
```

version은 필요한 경우만 사용한다.

Published Dependency에는 Version Pinning을 권장한다.

13. Principal Reference

Actor Reference:

```
{  
  "type": "user",  
  "id": "usr_123"  
}
```

지원 유형:

user

service

workload

agent

system

14. Version

Version은 Published Specification을 나타낸다.

조건:

- integer ≥ 1
- Publish 시 증가

- Published Version 수정 금지
-

15. Revision

Revision은 Draft 수정 번호다.

조건:

- integer ≥ 1
 - Draft Update 시 증가
 - Optimistic Concurrency에 사용 가능
-

16. ETag

외부 API에서는 Revision과 별도로 ETag를 제공할 수 있다.

예:

"agt_123-r17"

ETag 내부 구성은 Contract가 아니다.

17. Lifecycle State

모든 Resource에 동일 State를 강제하지 않는다.

다만 공통 State Vocabulary는 다음을 사용한다.

DRAFT

VALIDATING

REVIEW

APPROVED

PUBLISHED

ACTIVE

SUSPENDED

DEPRECATED

ARCHIVED

DELETED

Domain이 추가 상태를 정의할 수 있다.

18. Specification

specification은 Resource의 Desired Definition이다.

사용자/관리자/API가 수정하는 대상이다.

Published Version에서는 Immutable.

19. Status

status는 Runtime 또는 Platform이 관리하는 Observed State다.

Application이 일반 Update API를 통해 직접 변경해서는 안 된다.

20. Labels

Labels는 다음 용도로 사용한다.

- Environment
- Team
- Project
- Cost Center
- Category

예:

```
{  
  "environment": "production",  
  "team": "platform"
```

}

Security Permission의 단독 근거로 사용하지 않는다.

21. Annotations

Annotations는 비실행 Metadata다.

예:

- description URL
- migration note
- ticket reference

Runtime 행동을 Annotation에 숨겨 정의하지 않는다.

22. Timestamp

모든 Canonical Timestamp는 UTC ISO 8601을 사용한다.

예:

2026-08-21T04:00:00Z

23. Enum 규칙

Enum 값은 UPPER_SNAKE_CASE를 권장한다.

예:

READ_ONLY

IRREVERSIBLE_WRITE

Public API에서는 Contract가 확정되면 임의 변경하지 않는다.

24. Nullable와 Missing

다음은 서로 다르다.

```
{  
  
  "value": null  
  
}
```

Schema에서 두 의미를 명시한다.

25. Unknown Fields

Canonical Resource는 기본적으로 Unknown Field를 Validation Error로 처리하는 것을 권장한다.

Extension Point는 별도 extensions 또는 metadata 영역에서 제공한다.

26. Tenant Resource

Tenant는 AGEX Multi-Tenant Architecture의 최상위 논리 경계다.

기본 구조:

```
{  
  
  "kind": "Tenant",  
  
  "metadata": {},  
  
  "specification": {},  
  
  "status": {}  
  
}
```

27. Tenant Specification

필드	타입	필수
display_name	string	Y
home_region	string	Y

필드	타입	필수
isolation_profile	enum	Y
data_residency_policy_id	string/null	N
default_model_profile_id	string/null	N
maximum_autonomy_level	enum	Y
quota_profile_id	string/null	N
retention_profile_id	string/null	N

28. Tenant Isolation Profile

공식 값:

SHARED

ISOLATED_DATA

ISOLATED_RUNTIME

DEDICATED

29. Tenant Status

필드	타입
state	enum
provisioning_state	string/null
runtime_health	string/null
data_plane_region	string/null
suspension_reason	string/null

공식 Tenant State:

PROVISIONING

ACTIVE

RESTRICTED

SUSPENDED

CLOSING

DELETING

DELETED

30. User Resource

User는 Platform Identity다.

최소:

id

display_name

primary_identity_provider

state

User가 Tenant 권한을 직접 포함하지 않는다.

Tenant 권한은 Membership Resource가 관리한다.

31. Membership Resource

Membership:

User ↔ Tenant

관계를 표현한다.

Specification:

필드	타입
----	----

user_id	string
---------	--------

필드	타입
----	----

tenant_id	string
-----------	--------

roles	array<string>
-------	---------------

state	enum
-------	------

expires_at	timestamp/null
------------	----------------

32. Membership State

INVITED

ACTIVE

SUSPENDED

REVOKED

EXPIRED

33. Service Identity

Application 또는 Backend Workload가 사용하는 Identity다.

Specification:

- name
- owner
- tenant scope
- permission bindings
- expiration policy

사람 계정 Credential을 서버 자동화에 사용하지 않는다.

34. Agent Resource

Agent는 AGEX의 기본 AI 실행 주체다.

최상위 구조:

```
{  
  "kind": "Agent",  
  "metadata": {},  
  "specification": {  
    "role": {},  
    "goal": {},  
    "instructions": [],  
    "capabilities": [],  
    "permissions": [],  
    "autonomy": {},  
    "model_policy": {},  
    "context_policy": {},  
    "knowledge": {},  
    "memory": {},  
    "tools": [],  
    "planning": {},  
    "collaboration": {},  
    "execution": {},  
    "evaluation": {}  
  },  
  "status": {}  
}
```

35. Agent Role


```
{  
  "name": "coordinator",  
  "description": "..."  
}
```

Role은 Permission을 부여하지 않는다.

36. Agent Goal

```
{  
  "primary": "...",  
  "completion_criteria": []  
}
```

Completion Criteria 예:

```
{  
  "type": "OUTPUT_SCHEMA_VALID"  
}
```

또는:

```
{  
  "type": "CUSTOM_VALIDATOR",  
  "validator_id": "..."  
}
```

37. Agent Instruction

Instruction는 구조화된 Array로 관리할 수 있다.

```
{  
  "id": "ins_1",
```

```
"type": "CORE",  
"content": "..."  
}
```

User Input과 Core Instruction을 동일 Array에 혼합하지 않는다.

38. Agent Capability

예:

MODEL_GENERATE

KNOWLEDGE_RETRIEVE

MEMORY_RECALL

MEMORY_PROPOSE

TOOL_CALL

WORKFLOW_START

AGENT_DELEGATE

ARTIFACT_CREATE

EVENT_EMIT

39. Agent Permission Binding

예:

```
{  
  "action": "plugin.action.execute",  
  "resource": {  
    "type": "PluginAction",  
    "id": "pla_xxx"  
  }  
}
```

```
}
```

Wildcard는 최소화한다.

40. Agent Autonomy

```
{
```

```
  "level": "L2"
```

```
}
```

공식 Enum:

L0

L1

L2

L3

L4

L5

Effective Level은 Tenant/Platform Policy와 교집합으로 결정한다.

41. Agent Model Policy

```
{
```

```
  "profile_id": "mdp_123",
```

```
  "required_capabilities": [
```

```
    "STRUCTURED_OUTPUT",
```

```
    "TOOL_CALLING"
```

```
  ]
```

```
}
```

Agent가 Provider Secret이나 Endpoint를 직접 포함하지 않는다.

42. Agent Knowledge Binding

```
{  
  "collections": [  
    {  
      "id": "knc_123",  
      "version": 2  
    }  
  ],  
  "max_results": 10  
}
```

43. Agent Memory Policy

```
{  
  "read_scopes": [  
    "SESSION",  
    "AGENT"  
  ],  
  "write_mode": "PROPOSAL"  
}
```

Write Mode:

NONE

PROPOSAL

POLICY_GOVERNED

44. Agent Tool Binding

```
{  
  "tool_id": "tool_123",  
  "enabled": true  
}
```

Tool Definition은 별도 Registry Resource를 참조한다.

45. Agent Planning

```
{  
  "mode": "DYNAMIC",  
  "max_steps": 10,  
  "max_depth": 3  
}
```

Mode:

NONE

STATIC

DYNAMIC

HIERARCHICAL

RECEDING_HORIZON

MULTI_AGENT

46. Agent Collaboration

```
{  
  "delegation_enabled": true,  
  "allowed_agents": [],
```

```
"max_delegation_depth": 2,  
"max_child_executions": 5  
}
```

47. Agent Execution Policy

```
{  
  "max_iterations": 20,  
  "timeout_seconds": 600,  
  "max_tool_calls": 20,  
  "budget_id": null  
}
```

48. Agent Evaluation Policy

```
{  
  "required_evaluation_suite_id": "eval_123",  
  "minimum_score": 0.8  
}
```

49. Agent Status

필드	타입
lifecycle_state	enum
active_version	integer/null
health	enum
last_execution_at	timestamp/null

필드	타입
last_evaluation_id	string/null
suspension_reason	string/null

50. Workflow Resource

Workflow는 Durable Execution Graph다.

Specification:

input_schema
steps
edges
variables
retry_policy
timeout_policy
output_schema

51. Workflow Step

공통 구조:

```
{  
  "id": "step_1",  
  "type": "AGENT",  
  "name": "Analyze",  
  "input": {},  
  "configuration": {}  
}
```

52. Workflow Step Type

공식 Core Step Type:

AGENT

MODEL

PLUGIN

FUNCTION

CONDITION

SWITCH

PARALLEL

JOIN

LOOP

WAIT

EVENT_WAIT

APPROVAL

SUB_WORKFLOW

EMIT_EVENT

END

53. Workflow Edge

```
{  
  "from": "step_a",  
  "to": "step_b",  
  "condition": null  
}
```

54. Workflow Variables

각 Variable에는:

- name
- schema
- scope
- initial value optional

을 정의한다.

Secret Value는 Workflow Variable에 저장하지 않는다.

55. Workflow Retry Policy

공통 RetryPolicy를 참조한다.

```
{  
  "max_attempts": 3,  
  "strategy": "EXPONENTIAL",  
  "initial_delay_ms": 500,  
  "max_delay_ms": 10000,  
  "jitter": true  
}
```

56. Workflow Timeout Policy

```
{  
  "execution_timeout_seconds": 3600,  
  "step_timeout_seconds": 300  
}
```

Step별 Override 가능.

57. Workflow Status

- lifecycle_state
- active_version
- validation_state
- last_execution
- suspension_reason

Execution State와 혼합하지 않는다.

58. Execution Resource

Execution은 실제 Runtime Instance다.

Execution은 Immutable Definition Resource와 다르게 Runtime State 중심 Resource다.

59. Execution Specification

```
{  
  "target": {  
    "type": "Agent",  
    "id": "agt_123",  
    "version": 4  
  },  
  "input": {},  
  "execution_policy": {},  
  "parent_execution_id": null,  
  "correlation_id": "cor_123"  
}
```

60. Execution Status

```
{  
  "state": "RUNNING",  
  "waiting_reason": null,  
  "current_task_id": "tsk_123",  
  "attempt": 1,  
  "started_at": "...",  
  "completed_at": null,  
  "result": null,  
  "error": null,  
  "usage": {},  
  "checkpoint_id": null  
}
```

61. Execution State

공식:

CREATED

VALIDATING

QUEUED

RUNNING

WAITING

RETRYING

COMPLETED

FAILED

CANCELLED

TIMED_OUT

TERMINATED

62. Waiting Reason

예:

APPROVAL

USER_INPUT

EVENT

TIME

CHILD_EXECUTION

DEPENDENCY

State를 무한정 세분화하지 않는다.

63. Parent / Child Execution

Multi-Agent 또는 Sub-Workflow는:

parent_execution_id

로 연결한다.

Tree Depth 제한은 Runtime Policy에서 관리한다.

64. Execution Snapshot

실행 시 별도 Snapshot Reference를 생성한다.

포함:

- Agent/Workflow Version
- Model Policy Version

- Tool Binding
- Permission Reference
- Runtime Policy
- Governance Policy

Snapshot은 재현성과 Audit에 사용한다.

65. Task Resource

Task는 Runtime Scheduler가 실행하는 최소 Dispatch 단위다.

Specification:

execution_id

task_type

payload_reference

required_worker_capability

priority

66. Task Status

CREATED

QUEUED

LEASED

RUNNING

SUCCEEDED

FAILED

RETRY_WAIT

CANCELLED

Task State와 Execution State는 다르다.

67. Task Lease

Task에는 필요 시:

- lease_owner
- lease_expires_at
- heartbeat_at

을 둔다.

Worker 장애를 감지하는 데 사용한다.

68. Idempotency

Side-effect Task에는:

idempotency_key

또는:

logical_action_id

가 필요할 수 있다.

69. Operation Resource

관리형 비동기 작업이다.

예:

- Export
 - Import
 - Reindex
 - Migration
-

70. Operation State

QUEUED

RUNNING

COMPLETED

FAILED

CANCELLED

Execution과 별도 Resource Type이다.

71. Knowledge Collection

Specification:

필드	타입
description	string
classification	enum
retrieval_policy	object
source_ids	array
retention_policy_id	string/null

72. Knowledge Source

Specification:

source_type

connector_id

configuration_reference

classification

sync_policy

Credential Value는 저장하지 않는다.

73. Knowledge Source Type

예:

FILE

API

DATABASE

OBJECT_STORAGE

CONNECTOR

74. Knowledge Document

주요 Metadata:

- document_id
 - source_id
 - source_version
 - title
 - classification
 - ACL
 - checksum
 - ingestion status
-

75. Knowledge Chunk

Chunk는 Derived Resource로 취급할 수 있다.

필드:

- document_id
- chunk_index

- text/artifact ref
- embedding ref
- metadata

Chunk 자체가 공식 원본은 아니다.

76. Knowledge Provenance

반드시 추적 가능해야 한다.

Chunk

→ Document

→ Source

77. Knowledge Classification

공식:

PUBLIC

INTERNAL

CONFIDENTIAL

RESTRICTED

78. Memory Resource

Memory Specification:

memory_type

scope

owner

content

confidence

importance

retention

source_execution

79. Memory Type

공식 Vocabulary:

WORKING

SESSION

AGENT

USER

TENANT

EPISODIC

SEMANTIC

PROCEDURAL

Type와 Scope가 동일 개념은 아니다.

80. Memory Scope

EXECUTION

SESSION

AGENT

USER

TENANT

81. Memory Lifecycle

PROPOSED

VALIDATING

ACTIVE

CONFLICTED

SUPERSEDED

EXPIRED

ARCHIVED

DELETED

82. Memory Content

Memory Content는 가능한 구조화한다.

예:

```
{  
  "subject": "...",  
  "predicate": "...",  
  "value": "..."  
}
```

자유 텍스트도 지원할 수 있으나 Metadata를 함께 유지한다.

83. Memory Retention

필드:

- expires_at
 - retention_policy_id
 - delete_propagation_status
-

84. Memory Deletion

Deleted Memory는 Retrieval Candidate에서 즉시 제외한다.

Derived Index/Delete Queue 상태를 별도로 추적할 수 있다.

85. Plugin Definition

Plugin은 Platform/Marketplace 기술 Package다.

Specification:

publisher

package

version

manifest

capabilities

actions

permissions

network

runtime

compatibility

86. Plugin Capability

예:

```
{  
  "name": "items",  
  "description": "..."  
}
```

Capability 자체는 실행 함수가 아니다.

87. Plugin Action

Action Schema:

필드	타입
id	string
name	string
input_schema	schema ref
output_schema	schema ref
required_permissions	array
side_effect	enum
timeout_policy	object/ref
idempotency_mode	enum

88. Plugin Side Effect

공식:

READ_ONLY

REVERSIBLE_WRITE

IRREVERSIBLE_WRITE

PRIVILEGED_ACTION

89. Plugin Network

Manifest:

```
{  
  "egress": [  
    {
```

```
    "host": "api.example.com",  
    "port": 443,  
    "protocol": "HTTPS"  
  }  
]  
}
```

90. Plugin Installation

Tenant-owned Resource.

Specification:

plugin_id

plugin_version

configuration

credential_bindings

enabled_actions

91. Plugin Installation Status

INSTALLING

INSTALLED

ENABLED

DISABLED

FAILED

SUSPENDED

UNINSTALLED

AUTHORIZED는 Lifecycle State가 아니라 Runtime Permission Decision이다.

92. Credential Binding

```
{  
  "name": "primary",  
  "secret_ref": "sec_123"  
}
```

Secret Value는 포함하지 않는다.

93. Model Provider Resource

Provider는 Platform 또는 Tenant BYOK Scope를 가질 수 있다.

Specification:

provider_type

provider_class

endpoint

credential_ref

regions

capabilities

rate_limits

94. Provider Class

공식:

PRIVATE

DIRECT_APPROVED

ENTERPRISE_APPROVED

AGGREGATOR

RESTRICTED

BLOCKED

95. Model Resource

Specification:

필드	타입
provider_id	string
external_model_id	string
capabilities	array
context_window	integer
input_modalities	array
output_modalities	array
tool_calling	boolean
structured_output	boolean
streaming	boolean
region	array
privacy_level	string
security_level	string
cost_profile_id	string/null
fallback_group	string/null

96. Model Capability

공식 기본 Vocabulary:

TEXT_GENERATION

STRUCTURED_OUTPUT

TOOL_CALLING

VISION

EMBEDDING

RERANKING

STREAMING

97. Model Status

REGISTERED

VALIDATING

EVALUATING

APPROVED

ACTIVE

DEGRADED

UNAVAILABLE

DEPRECATED

BLOCKED

98. Model Profile

Tenant/Agent가 사용하는 Routing Policy Resource다.

Specification:

required_capabilities

allowed_providers

denied_providers

allowed_provider_classes

region_constraints

quality_profile

latency_profile

cost_budget

fallback_policy

99. Policy Resource

Policy는 Machine-readable Rule이다.

Specification:

scope

effect

subjects

actions

resources

conditions

obligations

priority

100. Policy Effect

ALLOW

DENY

REQUIRE_APPROVAL

단, Platform Security Baseline의 DENY는 하위 ALLOW로 Override할 수 없다.

101. Policy Scope

PLATFORM

ENVIRONMENT

TENANT

RESOURCE

102. Policy Condition

가능한 Context:

- Actor
 - Tenant
 - Resource
 - Data Classification
 - Risk
 - Region
 - Time
 - Cost
 - Side Effect
-

103. Policy Obligation

Permit에 추가 조건을 붙일 수 있다.

예:

AUDIT_REQUIRED

MASK_OUTPUT

APPROVAL_REQUIRED

LIMIT_RATE

104. Approval Resource

Specification:

execution_id

action_reference

target_reference

payload_digest

risk_level

requested_by

approver_policy

expires_at

105. Approval State

PENDING

APPROVED

REJECTED

EXPIRED

REVOKED

106. Approval Binding

Approval은 반드시 다음 중 하나 이상을 Binding한다.

- payload digest
- action id
- resource version
- execution id

단순 "앞으로 허용" 형태의 Approval을 지양한다.

107. Risk Level

공식:

LOW

MODERATE

HIGH

CRITICAL

108. Event Envelope

모든 Domain Event는 공통 Envelope를 사용한다.

```
{  
  "event_id": "evt_123",  
  "event_type": "execution.completed",  
  "event_version": 1,  
  "occurred_at": "...",  
  "scope_type": "TENANT",  
  "tenant_id": "ten_123",  
  "actor": {},  
  "resource": {},  
  "correlation_id": "cor_123",  
  "data": {}  
}
```

109. Event ID

Event는 immutable이며 고유 ID를 가진다.

Consumer Deduplication에 사용할 수 있다.

110. Event Version

event_version은 Event Payload Contract Version이다.

Resource Version과 별개다.

111. Command Envelope

필요한 내부 Command도 별도 표준을 둘 수 있다.

command_id

command_type

target

actor

tenant

payload

Event와 Command를 혼용하지 않는다.

112. Audit Event

Audit 최소 필드:

audit_id

timestamp

actor

tenant

action

resource

result

reason_code

request_id

correlation_id

113. Audit Result

SUCCESS

DENIED

FAILED

114. Audit Sensitive Data

Audit에 다음을 저장하지 않는다.

- Secret
- Password
- Raw Token
- Raw CoT

필요하면 Hash/Digest/Reference 사용.

115. Usage Record

최소:

usage_id

tenant_id

execution_id

resource

usage_type

quantity

unit

provider

occurred_at

idempotency_key

116. Usage Type

예:

MODEL_INPUT_TOKENS

MODEL_OUTPUT_TOKENS

RUNTIME_DURATION

PLUGIN_CALL

KNOWLEDGE_STORAGE

KNOWLEDGE_RETRIEVAL

117. Cost Record

Usage와 Cost를 분리할 수 있다.

usage_id

cost_amount

currency

cost_basis

provider

118. Money Type

Float를 사용하지 않는다.

권장:

```
{  
  "amount": "12.3400",  
  "currency": "USD"  
}
```

119. Artifact Resource

Specification:

media_type

size

checksum

storage_ref

classification

retention

120. Artifact Access

Artifact URL 자체가 Permission이 아니다.

다운로드 시 Tenant/Resource Permission을 다시 확인한다.

121. Common Retry Policy

공식 구조:

```
{  
  "max_attempts": 3,  
  "strategy": "EXPONENTIAL",  
  "initial_delay_ms": 500,
```

```
"max_delay_ms": 30000,  
"jitter": true,  
"retryable_codes": []  
}
```

122. Retry Strategy

NONE

FIXED

LINEAR

EXPONENTIAL

123. Common Timeout Policy

```
{  
  "timeout_ms": 30000  
}
```

Long-running Execution에서는 Runtime-level Timeout을 별도 적용한다.

124. Cancellation Policy

예:

IMMEDIATE_IF_SAFE

COOPERATIVE

CHECKPOINT_THEN_CANCEL

NOT_CANCELLABLE_AFTER_SIDE_EFFECT

125. Error Envelope

공식 API Error:

```
{  
  "error": {  
    "code": "POLICY_DENIED",  
    "category": "POLICY",  
    "message": "The action is not permitted.",  
    "request_id": "req_123",  
    "correlation_id": "cor_123",  
    "details": {}  
  }  
}
```

126. Error Category

공식:

AUTHENTICATION

AUTHORIZATION

POLICY

VALIDATION

NOT_FOUND

CONFLICT

QUOTA

RATE_LIMIT

RUNTIME

PROVIDER

TIMEOUT

INTERNAL

127. Pagination

Canonical Cursor Response:

```
{  
  "data": [],  
  "next_cursor": "cur_123",  
  "has_more": true  
}
```

128. API Request Metadata

필수/권장 Header:

Authorization

X-AGEX-Tenant

X-Request-ID

X-Correlation-ID

Idempotency-Key

If-Match

Endpoint 성격에 따라 적용.

129. Tenant Context Schema

```
{  
  "tenant_id": "ten_123",  
  "scope_type": "TENANT"  
}
```

Runtime에서는 Authenticated Principal과 교차 검증한다.

130. Authentication Context

내부 Context:

principal_id

principal_type

authentication_method

session_id

credential_id

authentication_time

131. Authorization Decision

구조화 Record:

decision

subject

action

resource

policy_ids

reason_code

obligations

132. Authorization Decision 결과

ALLOW

DENY

CONDITIONAL

Conditional은 Approval 등의 Obligation을 가질 수 있다.

133. Data Classification Context

모든 민감 Data Flow에서 다음을 전달할 수 있다.

classification

origin

handling_policy

Agent가 Classification을 임의로 낮출 수 없다.

134. Schema Validation 단계

API:

Transport Parse

→ Schema Validation

→ Authentication

→ Authorization

→ Domain Validation

실제 순서는 보안/성능 구현에 따라 일부 조정 가능하지만 Schema와 Domain Validation을 구분한다.

135. Domain Validation

예:

Agent:

- Model Profile 존재
- Tool Binding 유효
- Maximum Autonomy 준수

Workflow:

- Graph Reachable
- Loop Bound 존재
- Dependency Version 유효

136. Publish Validation

Published Version 생성 전 최소:

Schema

References

Permissions

Security

Dependencies

Lifecycle

Domain Rules

을 검증한다.

137. Activation Validation

Active 전 추가:

Owner

Evaluation

Runtime Compatibility

Required Infrastructure

Governance Approval

이 필요할 수 있다.

138. Dependency Graph

Resource Dependencies는 명시적으로 추적한다.

예:

Workflow v4

└─ Agent v3

└─ Plugin v2

└─ Model Profile v7

139. Dependency Reference

Published Resource에서는 정확한 Version Pinning을 우선한다.

140. Circular Dependency

Publish 단계에서 Resource Dependency Cycle을 검증한다.

허용되는 Runtime Cycle과 Definition Cycle을 구분한다.

141. Database Logical Model

DB는 Canonical Resource를 저장하기 위한 구현이다.

권장 논리 Table Group:

resources

resource_versions

resource_revisions

executions

tasks

events

audit_events

usage_records

Domain별 상세 Table을 둘 수 있다.

142. Resource Table

공통 Metadata 저장 가능:

id

kind

scope_type

tenant_id

name

current_revision

active_version

lifecycle_state

created_at

updated_at

143. Resource Version Table

Published Immutable Specification:

resource_id

version

specification

schema_version

checksum

published_at

published_by

144. Draft Revision Table

필요 시 Draft 변경 기록:

resource_id

revision

specification

updated_at

updated_by

145. JSON vs Normalized Table

모든 Specification을 무조건 JSON 한 Column에 저장하거나, 반대로 모든 필드를 무조건 Column으로 쪼개는 규칙을 두지 않는다.

기준:

- Query 필요성
- Constraint
- Performance
- Domain Ownership

Canonical Schema는 Storage Shape와 독립적이다.

146. Foreign Key 정책

동일 DB Boundary 내부에서는 FK를 사용할 수 있다.

하지만 Domain 간 강한 결합이 Architecture Boundary를 깨지 않도록 한다.

147. Soft Delete

일부 Resource에서는 Soft Delete를 사용할 수 있다.

그러나 Privacy/Delete Requirement가 있는 Content를 영구 Soft Delete만으로 처리해서는 안 된다.

148. Derived Store

다음은 Derived Data다.

- Search Index
- Vector Index
- Cache
- Analytics Projection

Source of Truth가 아니다.

149. Deletion Propagation

삭제 Flow:

Source of Truth Delete

→ Retrieval Block

→ Search Delete

→ Vector Delete

→ Cache Invalidate

→ Verification

150. Machine-readable Contract 생성 순서

실제 개발에서는 이 문서를 기반으로 먼저 다음 파일을 만든다.

common/resource-metadata.schema.json

tenant/tenant.schema.json

agent/agent.schema.json

workflow/workflow.schema.json
runtime/execution.schema.json
runtime/task.schema.json
knowledge/collection.schema.json
memory/memory.schema.json
plugin/plugin.schema.json
model/model.schema.json
governance/policy.schema.json
governance/approval.schema.json
common/event-envelope.schema.json

151. 우선 구현 Schema

Phase 0에서 반드시 먼저 만들어야 하는 Schema:

1. ResourceMetadata
 2. ResourceReference
 3. TenantContext
 4. PrincipalReference
 5. ErrorEnvelope
 6. EventEnvelope
 7. RetryPolicy
 8. TimeoutPolicy
-

152. Phase 1 Schema

- Tenant
- User

- Membership
 - ServiceIdentity
 - Policy
-

153. Phase 2 Schema

- Execution
 - Task
 - Operation
 - Checkpoint Reference
-

154. Phase 3 Schema

- Provider
 - Model
 - ModelProfile
 - ModelInvocation Result
-

155. Phase 4 Schema

- Agent
 - ToolDefinition
 - ActionCandidate
 - AgentPlan
-

156. Phase 5 Schema

- Workflow
- WorkflowStep

- WorkflowEdge
 - WorkflowVariable
-

157. Phase 6 Schema

- KnowledgeCollection
 - KnowledgeSource
 - KnowledgeDocument
 - Memory
-

158. Phase 7 Schema

- Plugin
 - PluginAction
 - PluginInstallation
 - CredentialBinding
-

159. Contract Test

모든 Schema에 다음 테스트를 둔다.

- Valid fixture accepted
 - Required missing rejected
 - Invalid enum rejected
 - Unknown critical field rejected
 - Cross-version compatibility tested
-

160. Golden Fixtures

/specs/fixtures/를 두는 것을 권장한다.

예:

valid-agent.json

invalid-agent-missing-goal.json

valid-execution.json

valid-plugin.json

161. SDK Type Generation

TypeScript:

Schema

→ Generated interface/type

Python:

Schema

→ Generated model/dataclass/pydantic-compatible type

구현 방식은 선택 가능하지만 동일 Contract를 사용한다.

162. Console Form Generation

일부 Form은 Schema Metadata를 활용할 수 있다.

하지만 UX 요구와 Security Control을 Schema만으로 모두 자동 생성하려 하지는 않는다.

163. Schema Extension

Third-party Plugin이 Core Schema에 임의 Field를 추가하지 못하도록 한다.

Extension Namespace 또는 Plugin-owned Schema를 사용한다.

164. Custom Metadata

확장이 필요한 경우:

```
{  
  "extensions": {  
    "publisher.example/key": {}  
  }  
}
```

형태를 검토할 수 있다.

Core Field 이름과 충돌하지 않아야 한다.

165. Compatibility 규칙

Backward-compatible:

- Optional Field 추가
- 새 Enum을 소비자가 Unknown 처리 가능하도록 설계된 경우
- 새 Endpoint

Breaking:

- Field 삭제
 - Type 변경
 - Required 변경
 - 기존 의미 변경
-

166. Schema Deprecation

Deprecated Field는:

deprecated = true

replacement = ...

removal_version = ...

같은 Metadata를 둘 수 있다.

167. Validation Error Path

Validation 오류는 정확한 Path를 반환한다.

예:

specification.autonomy.level

168. Resource Checksum

Published Specification에는 Checksum을 기록하는 것을 권장한다.

용도:

- Integrity
 - Snapshot
 - Audit
 - Package Verification
-

169. Immutable Version Enforcement

DB와 Service Layer 모두 Published Version Update를 차단해야 한다.

UI에서만 수정 버튼을 숨기는 것으로 끝내지 않는다.

170. Schema Security Rule

Schema가 Secret Type을 포함하는 경우 Raw Secret 필드를 Resource Schema에 넣지 않는다.

항상 SecretReference를 사용한다.

171. Secret Reference

Canonical:

```
{  
  "secret_id": "sec_123"  
}
```

필요하면:

version

purpose

추가 가능.

172. Payload Digest

Approval이나 Audit용 Payload Digest는 Canonical Serialization을 기준으로 생성해야 한다.

동일 의미 Payload가 다른 JSON 순서 때문에 다른 Digest가 되지 않도록 한다.

173. Canonical Serialization

특정 Signing/Hash가 필요한 Resource에는 JSON Canonicalization 규칙을 별도로 정의한다.

Marketplace Package와 Approval에서 중요하다.

174. Schema Acceptance Criteria

본 Specification이 실제 개발에 사용되기 위해 다음 조건을 충족해야 한다.

1. Common Resource Model이 존재한다.
2. Platform/Tenant Scope가 구분된다.
3. Version/Revision이 구분된다.
4. Specification/Status가 구분된다.
5. Tenant/User/Membership Model이 정의된다.
6. Agent Schema가 정의된다.

7. Workflow Schema가 정의된다.
8. Execution/Task가 분리된다.
9. Operation이 Execution과 분리된다.
10. Knowledge/Memory가 분리된다.
11. Plugin/Tool/Installation이 분리된다.
12. Provider/Model/Profile이 분리된다.
13. Policy/Approval/Risk가 정의된다.
14. Event Envelope가 정의된다.
15. Audit/Usage가 정의된다.
16. Error Contract가 정의된다.
17. Retry/Timeout 공통 Contract가 존재한다.
18. Secret Reference 원칙이 존재한다.
19. Schema Versioning이 존재한다.
20. Machine-readable Schema로 변환 가능하다.

175. 금지사항

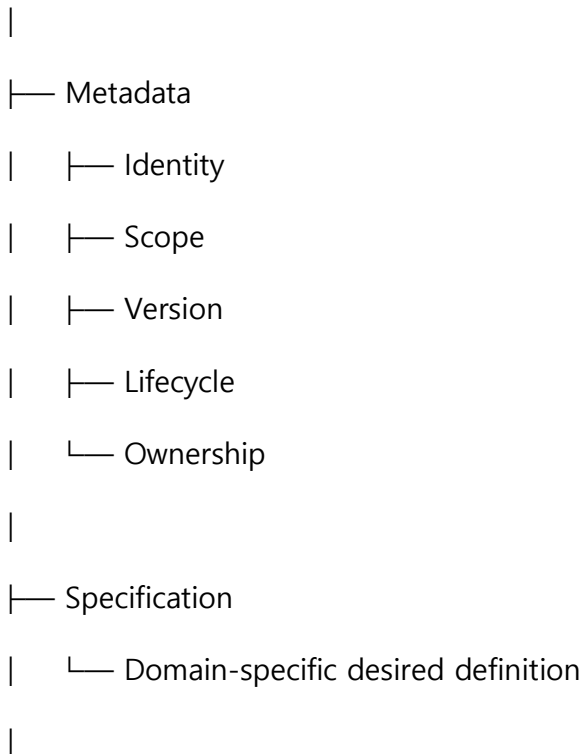
다음 Schema 설계를 사용하지 않는다.

- 모든 Resource를 하나의 data:any 구조로 저장
- Tenant ID 없는 Tenant Resource
- Published Version 직접 수정
- Agent Schema에 Provider API Key 저장
- Workflow 변수에 Secret 저장
- Plugin Definition에 Tenant Credential 저장
- Model과 Provider를 하나의 문자열로 표현
- Marketplace Entitlement를 Permission Field로 사용

- Knowledge와 Memory 동일 Resource Type 사용
- Execution과 Operation 동일 State Machine 강제
- Queue Message를 Execution Schema 자체로 사용
- Risk와 Data Classification 혼용
- Approval에 Payload Binding 없음
- Event에 Tenant/Resource Reference 없음
- Audit에 Raw Token 저장
- Cost를 Float로 저장
- Unknown Field를 아무 검증 없이 무제한 허용
- Console UI Form을 Schema Source of Truth로 사용
- DB Table을 API Schema로 직접 노출

176. Canonical Resource Model 최종 구조

AGEX Resource



└─ Status

└─ Platform/runtime observed state

177. Runtime Model 최종 구조

Resource Definition



Published Version



Execution Snapshot



Execution



Task



Worker



Result / Event / Audit / Usage

178. Agent 관련 최종 구조

Agent

└─ Goal

└─ Instruction

└─ Capability

└─ Permission

└─ Autonomy

- └─ Model Policy
 - └─ Knowledge
 - └─ Memory
 - └─ Tool Binding
 - └─ Planning
 - └─ Collaboration
 - └─ Execution Policy
-

179. Workflow 관련 최종 구조

Workflow

- └─ Input Schema
 - └─ Steps
 - └─ Edges
 - └─ Variables
 - └─ Retry
 - └─ Timeout
 - └─ Approval
 - └─ Output Schema
-

180. Plugin 관련 최종 구조

Plugin Package

- └─ Manifest
- └─ Capability
- └─ Actions
- └─ Permission Requirements

|— Side Effects

|— Network

└— Runtime

Tenant

└— Plugin Installation

 |— Configuration

 |— Credential Binding

 └— Enabled Actions

181. Model 관련 최종 구조

Provider

↓

Model

↓

Model Profile

↓

Model Gateway

↓

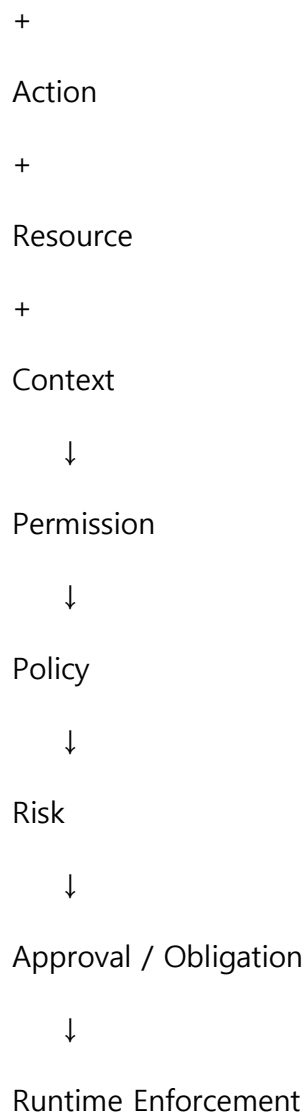
Router

↓

Execution

182. Governance 관련 최종 구조

Actor



183. 실제 개발에 착수하기 전 다음 작업

본 문서가 확정되면 바로 다음을 만들어야 한다.

1.

/specs/schemas/common/*.json

2.

Tenant / Agent / Execution 실제 JSON Schema

3.

OpenAPI 기본 골격

4.

TypeScript Type Generation

5.

Python Type Generation

6.

DB Logical ERD

7.

Contract Tests

184. 최종 정의

AGEX Canonical Resource & Schema Specification은 30종의 사람이 읽는 공식 문서에서 정의된

Product·Architecture·Runtime·Agent·Workflow·Knowledge·Memory·Plugin·Model·Security 개념을 실제 Software Contract로 변환하는 최상위 구현 명세이다.

AGEX의 모든 주요 구성요소는 명시적인 Resource 또는 Runtime Entity로 정의하고, Resource는 Metadata·Specification·Status 구조를 기본으로 하며 Platform/Tenant Scope, Version, Revision 및 Lifecycle을 명확하게 관리한다.

Agent, Workflow, Knowledge, Memory, Plugin 및 Model은 서로 다른 Domain Resource로 정의하며 편의를 이유로 하나의 범용 AI Object로 통합하지 않는다.

Agent와 Workflow Definition은 Published Version에서 Immutable하며 실행 시 정확한 Version과 Policy Reference를 Snapshot으로 고정함으로써 장기 실행의 재현성을 확보한다.

Execution과 Task는 분리하고 Queue를 Source of Truth로 사용하지 않으며 Durable Execution Store와 Task Lease를 통해 Worker 장애 이후에도 실행을 복구할 수 있어야 한다.

Plugin Package, Plugin Installation, Action 및 Agent Tool을 분리하고 Tenant Credential은 Plugin Definition이 아니라 Tenant Installation과 Secret Store에 귀속시킨다.

Provider, Model 및 Model Profile 역시 별도 Resource로 관리하여 Agent가 특정 Provider 구현에 종속되지 않도록 한다.

Permission, Policy, Risk, Approval 및 Entitlement는 각각 독립 개념으로 유지하며 Security 또는 Commercial 개념을 하나의 Boolean Permission으로 통합하지 않는다.

모든 API, Event, SDK 및 Console은 이 Canonical Contract에서 파생되어야 하며 Database Schema나 UI Form이 공식 Resource Definition을 역으로 결정해서는 안 된다.

본 문서는 다음 문장으로 최종 정의한다.

AGEX Canonical Resource & Schema Specification은 AGEX의 모든 Resource와 Runtime Entity를 코드·API·SDK·Event·DB·Test가 공유할 수 있는 명시적이고 Versioned 된 Contract로 정의하여, AGEX 공식 문서 체계를 실제 구현 가능한 Software Specification으로 전환하는 최상위 Schema 기준이다.